

PROJECT DOCUMENTATION

Mental Health in Tech: Data Analysis & Dashboard

Folorunso Julius Falana (JAYBITE98)

RN, MSc Public Health Nursing Candidate | BSc Data Science & AI Candidate

Near East University | IU International University of Applied Sciences

Repository: github.com/JAYBITE98/Mental-Health-Project-Analysis

Documentation Date: June 2026 | Language: Python | Tools: Pandas · Matplotlib · Power BI

1. Project Overview

This project is an end-to-end data analysis pipeline that investigates mental health in the technology workplace. Starting from a synthetically generated survey dataset, the project applies structured data cleaning, exploratory data analysis (EDA), and business intelligence visualisation to surface meaningful patterns in treatment-seeking behaviour among tech workers.

The workflow deliberately mirrors the real-world data analysis lifecycle — from raw, 'dirty' data to a polished, interactive Power BI dashboard — making the project a strong demonstration of both technical skill and applied data thinking in a public health context.

1.1 Objectives

- Generate a realistic survey dataset with intentional data quality issues (missing values, typos) to simulate a real-world scenario.
- Apply systematic data cleaning techniques using Python and Pandas.
- Conduct exploratory analysis to investigate key relationships (gender vs. treatment-seeking, work interference vs. treatment-seeking).
- Produce high-quality static visualisations using Matplotlib.
- Export a clean dataset and build an interactive, slicer-enabled dashboard in Power BI.

1.2 Key Finding

Key Insight: Individuals who reported that work "**sometimes**" interfered with their mental health were the most likely to seek treatment (45.5%), while those who reported it "**often**" interfered were the least likely (30.2%) — a counter-intuitive finding that warrants further investigation.

2. Tools & Technologies

Tool	Version / Type	Purpose in This Project
Python	3.x	Primary scripting language for all data processing and analysis tasks.
Pandas	Library	Data loading, cleaning, transformation, cross-tabulation, and export.
Matplotlib	Library	Generation of static bar charts for EDA and visual quality checks.
NumPy	Library	Random seed setting and NaN insertion during dataset generation.
Power BI Desktop	Dashboard Tool	Interactive dashboard with slicer-based filtering for end-user exploration.
CSV (comma-separated values)	Data Format	Storage and exchange of raw and cleaned datasets.
.pbix	Power BI Format	Packaged dashboard file for sharing and publishing.

3. Repository Structure

The repository contains the following files, organised in a logical data pipeline progression:

File / Folder	Type	Description
generate_data.py	Python	Generates the synthetic survey dataset (500 rows) with intentional data quality issues.
mental_health_survey.csv	Raw Data	Output of generate_data.py. Contains missing values and typos; serves as the project's raw input.
analysis.py	Python	Initial EDA script. Loads raw data, prints summary statistics, and renders a basic bar chart.
cleaned_analysis.py	Python	Full data-cleaning pipeline. Fixes typos, imputes missing ages, drops irrelevant columns, and saves a polished chart.
cleaned_mental_health_chart.png	Image	Exported bar chart from cleaned_analysis.py showing treatment-seeking distribution.
gender_analysis.py	Python	Cross-tabulation and grouped bar chart of treatment-seeking behaviour by gender.
gender_treatment_chart.png	Image	Exported grouped bar chart for gender vs. treatment analysis.

File / Folder	Type	Description
work_analysis.py	Python	Cross-tabulation and grouped bar chart of treatment-seeking by work interference level, including percentage breakdown.
work_treatment_chart.png	Image	Exported grouped bar chart for work interference vs. treatment analysis.
save_for_powerbi.py	Python	Applies all cleaning steps and exports cleaned_mental_health_data.csv for import into Power BI.
cleaned_mental_health_data.csv	Clean Data	Cleaned dataset ready for Power BI ingestion. All typos corrected, missing values imputed, irrelevant columns removed.
Mental_Health_Dashboard.pbix	Power BI	Final interactive dashboard with slicers for gender and work interference.
.idea/	IDE Config	PyCharm project configuration files (not required to run the project).

4. Dataset Description

4.1 Raw Dataset: mental_health_survey.csv

The raw dataset is generated programmatically by generate_data.py. It simulates a workplace mental health survey covering 500 respondents across a range of countries and company sizes. The dataset intentionally contains real-world data quality issues to make the cleaning pipeline realistic and instructive.

Column	Data Type / Values	Description
Timestamp	Date string	Simulated survey date (Jan 2024). Dropped during cleaning as non-analytical.
Age	Numeric (18–69) + NaN	Respondent age. 20 values intentionally set to NaN.
Gender	Male / Female / Non-binary	Gender identity. Slight over-representation of Male and Female.
Country	7 countries + NaN	Country of residence. 15 values intentionally set to NaN; dropped during cleaning.
family_history	Yes / No	Whether the respondent has a family history of mental illness (~40% Yes).
treatment	Yes / No / Y / N	Whether the respondent has sought professional mental health treatment. Contains 15 'Y' and 10 'N' typos.

Column	Data Type / Values	Description
work_interfere	Often / Rarely / Sometimes / Never + NaN	How often mental health interferes with work. 25 values intentionally NaN.
num_employees	5 size bands	Company size (1–5 up to 500+).
remote_work	Yes / No	Whether respondent works remotely.
tech_company	Yes / No	Whether respondent works in a tech company.
benefits	Yes / No / Don't know	Whether employer provides mental health benefits.
care_options	Yes / No / Not sure	Whether respondent knows about employer care options.
wellness_program	Yes / No / Don't know	Whether employer has a wellness program.
seek_help	Yes / No / Maybe	Whether the employer provides resources to seek mental health help.

Data quality issues introduced: 20 NaN values in Age, 15 NaN values in Country, 25 NaN values in work_interfere, 15 'Y' values and 10 'N' values in the treatment column (typos).

4.2 Cleaned Dataset: cleaned_mental_health_data.csv

After applying all cleaning steps, the dataset retains 500 rows and is reduced from 14 to 12 columns (Timestamp and Country removed). Missing Age values are imputed with the column mean. Typos in the treatment column are standardised to Yes/No. This cleaned file is the source of truth for Power BI.

5. Script Documentation

This section documents each Python script in execution order, describing its purpose, inputs, outputs, and key logic.

5.1 generate_data.py — Dataset Generation

Purpose

Creates a realistic, reproducible synthetic survey dataset with 500 rows. Intentional data quality issues are embedded for use as cleaning practice.

Input: None (generates data from scratch)

Output: mental_health_survey.csv

Lines of code: 68

Key Logic

- Sets NumPy and Python random seeds (42) for full reproducibility.
- Defines weighted option lists (biased toward Male/Female in gender; biased toward 'Rarely'/'Never' in work_interfere) to simulate realistic survey response distributions.
- Generates 500 rows using random.choice() with list comprehensions across 14 columns.
- Intentionally corrupts the dataset: 20 NaN values injected into Age, 15 into Country, 25 into work_interfere using random.sample() and df.loc[].
- Changes 15 'Yes' entries and 10 'No' entries in the treatment column to 'Y' and 'N' respectively, simulating real data entry inconsistencies.
- Saves the result with df.to_csv('mental_health_survey.csv', index=False).

```
# Example: Weighted gender distribution
genders = ['Male', 'Female', 'Non-binary', 'Male', 'Female']

# Example: Injecting missing values
age_nan_indices = random.sample(range(num_people), 20)
df.loc[age_nan_indices, 'Age'] = np.nan

# Example: Introducing typos
df.loc[random.sample(range(num_people), 15), 'treatment'] = 'Y'
```

5.2 analysis.py — Initial Exploratory Analysis

Purpose

Provides a first look at the raw dataset. Prints column names, the first 5 rows, and treatment value counts. Renders a basic bar chart of treatment-seeking behaviour — before any cleaning. This step exposes the 'Y' and 'N' typos as separate bars in the chart.

Input: mental_health_survey.csv

Output: Console printout + on-screen bar chart (not saved to file)

Lines of code: 40

Key Logic

- Loads data with pd.read_csv().
- Calls df.head(), df.columns.tolist(), and df['treatment'].value_counts() to profile the data.
- Uses Matplotlib to render a basic bar chart with four colour bars (one per unique treatment value: Yes, No, Y, N).
- Uses plt.show() only — chart is displayed on screen but not exported. This script is diagnostic, not production.

```
treatment_counts = df['treatment'].value_counts()
treatment_counts.plot(kind='bar', color=['skyblue', 'salmon', 'lightgreen',
'orange'])
plt.title('Have you sought treatment for mental health?')
plt.show() # Displays the chart without saving
```

5.3 cleaned_analysis.py — Data Cleaning & Cleaned Chart

Purpose

Core data cleaning script. Applies all necessary cleaning transformations to the raw dataset and produces a publication-ready bar chart of the cleaned treatment distribution.

Input: `mental_health_survey.csv`

Output: `cleaned_mental_health_chart.png` (300 DPI)

Lines of code: 72

Cleaning Steps Applied

#	Step	Technique	Rationale
1	Fix treatment typos	<code>df['treatment'].replace({'Y':'Yes','N':'No'})</code>	Standardise Yes/No responses to eliminate spurious categories in analysis.
2	Impute missing ages	<code>df['Age'].fillna(df['Age'].mean())</code>	Mean imputation preserves all 500 rows while replacing 20 NaN values.
3	Drop Timestamp column	<code>df.drop(columns=['Timestamp'])</code>	No analytical value; purely a metadata column.
4	Drop Country column	<code>df.drop(columns=['Country'])</code>	15 missing values and not used in the core analysis questions.

Chart Features

- Colour scheme: coral (#FF6B6B for No) and teal (#4ECDC4 for Yes).
- Value annotations placed above each bar using `plt.text()` for immediate readability.
- Saved with `dpi=300` and `bbox_inches='tight'` for print-quality output.

```
# Step 1: Fix typos
df['treatment'] = df['treatment'].replace({'Y': 'Yes', 'N': 'No'})

# Step 2: Impute missing ages with column mean
df['Age'] = df['Age'].fillna(df['Age'].mean())

# Step 3 & 4: Drop non-analytical columns
```

```
df = df.drop(columns=['Timestamp', 'Country'])

# Save high-resolution chart
plt.savefig('cleaned_mental_health_chart.png', dpi=300, bbox_inches='tight')
```

5.4 gender_analysis.py — Gender vs. Treatment Analysis

Purpose

Examines whether gender is associated with differences in mental health treatment-seeking behaviour. Uses a cross-tabulation and grouped bar chart to compare Yes/No treatment counts across Male, Female, and Non-binary respondents.

Input: [mental_health_survey.csv](#) (cleaned inline)

Output: [gender_treatment_chart.png](#) (300 DPI)

Lines of code: 46

Key Logic

- Applies the same three cleaning steps as `cleaned_analysis.py` (inline, not re-importing the cleaned CSV).
- Calls `df['Gender'].value_counts()` to show the distribution of gender responses.
- Uses `pd.crosstab(df['Gender'], df['treatment'])` to generate a frequency table of treatment-seeking by gender — a standard EDA technique for categorical-vs-categorical analysis.
- Plots the crosstab as a grouped bar chart with two bars per gender group (Yes / No).
- Applies numerical annotations per bar using a nested loop over crosstab rows and values.

```
# Cross-tabulation: Gender vs Treatment
gender_treatment = pd.crosstab(df['Gender'], df['treatment'])

# Grouped bar chart
gender_treatment.plot(kind='bar', figsize=(10, 6), color=['#FF6B6B', '#4ECDC4'])

# Annotate bars with counts
for i, row in enumerate(gender_treatment.iterrows()):
    for j, value in enumerate(row[1]):
        plt.text(i - 0.1 + j*0.2, value + 3, str(value), ha='center',
                fontweight='bold')
```

5.5 work_analysis.py — Work Interference vs. Treatment Analysis

Purpose

Investigates the relationship between the degree to which mental health interferes with work (Never / Rarely / Sometimes / Often) and the likelihood of seeking treatment. This analysis produced the project's most notable finding.

Input: `mental_health_survey.csv` (cleaned inline)

Output: `work_treatment_chart.png` (300 DPI) + console percentage table

Lines of code: 49

Key Logic

- Applies the standard cleaning pipeline.
- Generates a cross-tabulation of `work_interfere` vs. `treatment` using `pd.crosstab()`.
- Computes row-wise percentages using `work_treatment.div(work_treatment.sum(axis=1), axis=0) * 100`, enabling comparison across groups of different sizes.
- Prints both the raw counts table and the percentage table to the console.
- Renders a grouped bar chart with counts, annotated per bar.

```
# Cross-tabulation
work_treatment = pd.crosstab(df['work_interfere'], df['treatment'])

# Row-wise percentage breakdown
work_pct = work_treatment.div(work_treatment.sum(axis=1), axis=0) * 100
print(work_pct.round(1))

# Key finding from percentage output:
# Sometimes → 45.5% sought treatment (HIGHEST)
# Often → 30.2% sought treatment (LOWEST — counter-intuitive)
```

Interpretation: The counter-intuitive result ("Often" interference → lowest treatment rate) may reflect avoidance behaviour, stigma, or functional impairment making it harder to seek help. This is a hypothesis worth testing in a larger, validated study.

5.6 `save_for_powerbi.py` — Export Pipeline for Power BI

Purpose

A concise, pipeline-focused script that consolidates all cleaning steps into a single run and exports the fully cleaned dataset to `cleaned_mental_health_data.csv`, making it ready for import into Power BI Desktop.

Input: `mental_health_survey.csv`

Output: `cleaned_mental_health_data.csv`

Lines of code: 22

Key Logic

- Executes all three cleaning steps (typo correction, mean imputation, column removal) in sequential order.
- Saves the result using `df.to_csv('cleaned_mental_health_data.csv', index=False)` — without the row index to keep the file clean for BI tools.
- Prints a summary confirmation including row count and column count.

```
# All cleaning consolidated
df['treatment'] = df['treatment'].replace({'Y': 'Yes', 'N': 'No'})
df['Age'] = df['Age'].fillna(df['Age'].mean())
df = df.drop(columns=['Timestamp', 'Country'])

# Export for Power BI
df.to_csv('cleaned_mental_health_data.csv', index=False)
```

6. Power BI Dashboard

The final deliverable of the project is an interactive Power BI dashboard (`Mental_Health_Dashboard.pbix`) that enables business users and stakeholders to explore the cleaned data dynamically.

6.1 Dashboard Features

- Interactive Slicers: Users can filter all visuals by Gender and by Work Interference level in real time, without writing any code.
- Dynamic charts update simultaneously based on slicer selection, allowing cross-filtered exploration.
- The dashboard is built on `cleaned_mental_health_data.csv`, ensuring all visualisations reflect the cleaned, validated data.

6.2 Recommended Visuals to Include

- Bar chart: Overall treatment-seeking counts (Yes vs. No).
- Grouped bar chart: Treatment-seeking by gender.
- Grouped bar chart: Treatment-seeking by work interference level.
- Donut / pie chart: Distribution of company sizes (`num_employees`).
- Stacked bar: Remote work status vs. treatment-seeking.
- Card visuals: Total respondents, % who sought treatment.

7. How to Run the Project

7.1 Prerequisites

- Python 3.x installed.
- Required libraries: `pip install pandas matplotlib numpy`
- Power BI Desktop installed (for viewing / editing the .pbix file).

7.2 Execution Order

1. Clone the repository: `git clone https://github.com/JAYBITE98/Mental-Health-Project-Analysis.git`
2. Generate the raw dataset: `python generate_data.py`
3. Run initial EDA: `python analysis.py`
4. Clean data and save chart: `python cleaned_analysis.py`
5. Run gender analysis: `python gender_analysis.py`
6. Run work interference analysis: `python work_analysis.py`
7. Export for Power BI: `python save_for_powerbi.py`
8. Open `Mental_Health_Dashboard.pbix` in Power BI Desktop and connect it to `cleaned_mental_health_data.csv` if prompted.

8. Skills Demonstrated

Skill Category	Evidence in This Project
Data Generation	Synthetic dataset creation with <code>numpy/random</code> ; intentional quality corruption; reproducible seeds.
Data Cleaning	Typo correction via <code>.replace()</code> , mean imputation with <code>.fillna()</code> , column pruning with <code>.drop()</code> .
Exploratory Data Analysis	Value counts, crosstabulation, row-wise percentage normalisation, multi-group comparison.
Data Visualisation (Python)	Matplotlib bar and grouped-bar charts; colour design; value annotation; 300 DPI export.
Business Intelligence	Power BI dashboard with interactive slicers; CSV-to-dashboard pipeline.
End-to-End Pipeline Design	Full data lifecycle from generation → cleaning → EDA → export → dashboard.
Public Health Domain Knowledge	Framing of treatment-seeking behaviour as a public health outcome; interpretation of counter-intuitive findings.
Version Control	Project published on GitHub under JAYBITE98 portfolio; clean repository structure.

9. Limitations & Future Directions

9.1 Current Limitations

- The dataset is synthetically generated, which limits the ecological validity of findings. No real-world conclusions can be drawn.
- Mean imputation for Age introduces bias if missingness is not random. Multiple imputation (e.g., via `sklearn.impute.IterativeImputer`) would be more robust.
- The Country column is dropped entirely due to missing values; a partial-imputation or majority-fill strategy could preserve geographic insight.
- No statistical tests (e.g., chi-square) are applied to validate associations found in the crosstabs.

9.2 Recommended Enhancements

- Apply chi-square tests to formally assess whether gender and work_interfere are statistically associated with treatment-seeking.
- Replace mean imputation with KNN or model-based imputation for a more principled approach.
- Extend the EDA to include family_history, benefits, and remote_work as potential predictors.
- Develop a logistic regression model to predict treatment-seeking probability from demographic and workplace features.
- Replace synthetic data with a real-world dataset (e.g., the OSMI Mental Health in Tech Survey from Kaggle).
- Publish the Power BI dashboard to the Power BI Service for cloud-based access and sharing.

10. References

- McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference. [Pandas library]
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
- Harris, C. R., et al. (2020). Array programming with NumPy. Nature, 585, 357–362.
- Open Sourcing Mental Illness (OSMI). Mental Health in Tech Survey. <https://osmihelp.org/research>
- Microsoft Corporation. Power BI Desktop. <https://powerbi.microsoft.com>
- GitHub Repository: JAYBITE98/Mental-Health-Project-Analysis. <https://github.com/JAYBITE98/Mental-Health-Project-Analysis>